

D5 — Part 1 Retrospective (İsmail Kabak, 1901042652)

Project: Recipix v4.1 — recipe DSL with dimensional types **My role in Part 1:** lexer owner; primary author of the v4.1 design spec; end-to-end integration of lexer + parser into the CLI pipeline.

What is working

The design spec went through four full review-and-fix passes ($v1 \rightarrow v4.1$) with a “professor” Claude session before we wrote any code, and that paid for itself many times over. The §2 minimum-requirement gap (no control structures) was caught at v1 and fixed; the keyword/identifier clashes (**for** as both a step modifier and a loop keyword), the greedy **serves** `<expr>` ambiguity, the **quantity_of** operator-vs-call-form decision, and the **substitute** slot resolution rule were all locked down before my partner started on the parser. The lexer itself is small and deterministic: °C is correctly emitted as a single `UNIT_KW` token, the two-token quantity-literal rule (Decision #7) is enforced — `200g` glued together is rejected with a line-numbered error recommending the correct `200 g` form — and all 12 lexer self-tests pass. The shared `tokens.py` contract that I published before the parser started kept integration clean: a single source of truth for token-type strings, imported by both teams.

What is hard

Two things were genuinely difficult. The first was **knowing when the spec was done**: every review pass surfaced new corners (action verbs as reserved keywords, the **pinch** count ambiguity, trailing-comma policy in list literals) and it would have been easy to spec-engineer forever. Asking the professor session for a yes/no readiness check at v3 — and getting a clean “no, two more passes” — was what stopped me. The second was the late **domain pivot**: my partner and I started on a CSV-query language called DataScript, then switched to Recipix the day before the deadline because the dimensional type system gave much richer Sebesta material. Migrating cleanly required a real `git rm`, not just a file delete (a mistake I almost shipped); ensuring the parser team could import a stable token contract from day one of the new design; and rewriting sample programs in time. It worked, but it ate the buffer.

What I plan to fix in Part 2

Three concrete carry-overs. **First**, the lexer’s error messages are correct but not all uniform — we settled on `parse error at line {line}, col {col}`: ... for the parser, and the lexer should follow the same prefix instead of its current `Lexer error at line {line}`: **Second**, the `tests/test_lexer.py` file still uses script-style assertions rather than `unittest.TestCase`, so it prints

output but does not register as failures in the unittest discovery run; I want this rewritten as proper `TestCase` classes before Part 2 so the lexer contributes to the green-bar count. **Third**, the `PYTHONPATH=src python -m unittest` command surfaced in the tester-agent run is a real packaging nit; a `conftest.py` or `setup.cfg` line will fix it. None of these affect Part 1 correctness, but they will make Part 2's type-checker work — which I expect to be the hardest single deliverable in the whole project — easier to land cleanly.